

CODEFEST AD ASTRA 2026 -ETAPA 1.

Informe Técnico

1. Introducción.

El presente informe documenta las decisiones tomadas a lo largo de la elaboración de la entrega final. Además, describe el proceso de construcción de la base de conocimiento vectorial a partir del corpus documental provisto por ADL para los tres fenómenos del reto, cubriendo las etapas de extracción y limpieza de texto, la estrategia de chunking aplicada a cada tipo de fuente, los criterios de selección del encoder o encoders utilizados, y el tipo de índice FAISS empleado para la indexación. Cada decisión se justifica en función de las características particulares del corpus.

2. Preprocesamiento y extracción de texto.

2.1 XLSX

Para los datasets XLSX del AI Index diseñamos un proceso de extracción pensado para tablas haciendo uso de Pandas, cargamos cada Excel forzando todos los valores a texto (dtype=str), evitando que Pandas introdujera artefactos de conversión automática.

- **Encabezados limpios:** Normalizamos los nombres de las columnas.
- **Filtrado de celdas corruptas:** Al revisar los archivos, notamos que varias celdas venían dañadas de origen con textos de frases repetidas con tabulaciones (\t), saltos de línea (\n) o múltiples espacios. Para solucionarlo, detectamos la primera aparición de estas marcas y cortamos el valor ahí, evitando pasar datos contaminados (y excesivamente largos) al embedding.
- **Sin ruido:** Eliminamos las celdas vacías o nulas.

2.2 Procesamiento de los documentos PDF

El corpus contiene PDFs de características muy diferentes. Algunos incluían texto digital seleccionable, mientras que otros estaban formados por páginas escaneadas o imágenes. Por esta razón, no era suficiente aplicar un único método de extracción.

El primer paso para cada documento consistió en intentar extraer directamente el texto de todas sus páginas.

Para esto utilizamos la biblioteca pdfplumber, que permite leer el contenido textual almacenado dentro de los PDF. Este método es rápido y conserva razonablemente el orden de lectura del documento.

En cada página se revisaron aspectos como:

- La cantidad de texto recuperado.
- La proporción de caracteres legibles.
- La presencia y el tamaño de las imágenes.
- La posibilidad de que la página fuera un escaneo.

Cuando el texto digital era suficiente y legible, se conservaba sin aplicar OCR.

Sin embargo, algunas páginas no contenían texto digital o solo devolvían unos pocos caracteres. En estos casos fue necesario aplicar reconocimiento óptico de caracteres (OCR)

La página se convertía temporalmente en una imagen con una resolución de 220 DPI. Después, la imagen se procesaba con Tesseract utilizando los idiomas español e inglés.

El OCR no se aplicó a todos los documentos. Se utilizó únicamente cuando las características de la página indicaban que podía recuperar información adicional. Lo anterior permitió disminuir considerablemente el tiempo de procesamiento.

Cuando una página tenía texto digital y también era candidata para OCR, se comparaba la cantidad de información obtenida por ambos métodos. Al final, se conservaba la versión que recuperara el contenido más completo.

Limpieza y almacenamiento

Después de extraer el texto se realizó una limpieza básica:

- Eliminación de caracteres de control.
- Normalización de caracteres Unicode.
- Corrección de espacios repetidos.
- Unión de líneas pertenecientes al mismo párrafo.
- Conservación de las separaciones entre párrafos.

En el caso de las fuentes con formato HTML, se estructuró un flujo de extracción cuyo objetivo primordial fue capturar el contenido legible y visible. Este proceso descartó cualquier componente de código o estructural que pudiese viciar la calidad de la representación vectorial.

Para llevar a cabo esta tarea, se empleó la librería BeautifulSoup, siguiendo estas directrices:

- **Depuración de boilerplate:** Se eliminaron secciones de cabeceras, pies de página y menús de navegación, omitiendo así elementos ornamentales carentes de relevancia semántica.

- **Jerarquía informativa:** Se respetó el orden natural de los elementos textuales, recorriendo encabezados, párrafos y listas, asegurando la integridad de la secuencia lógica planteada por el autor original.

Respecto a la **limpieza y normalización** del texto extraído, se aplicaron los siguientes criterios:

- Transformación de entidades HTML a sus correspondientes caracteres reales.
- Supresión de etiquetas residuales y caracteres técnicos de control.
- Estandarización rigurosa de la codificación bajo el estándar UTF-8.
- Simplificación de espacios y tabulaciones múltiples en un único espacio en blanco.
- Mantenimiento de los saltos de línea para preservar la estructura de los párrafos.

El corpus integra un total de 964 archivos JSON procedentes de diversas entidades y centros de pensamiento (CSIS, Atlantic Council, SIPRI, CENIA, entre otros). Ante la ausencia de un esquema uniforme, se implementó un extractor versátil capaz de decodificar la organización interna propia de cada documento.

El flujo de trabajo permitió distinguir dos categorías principales de datos:

1. **Informes y análisis estructurados:** Se priorizó la extracción del núcleo narrativo (párrafos de cuerpo, resúmenes y alertas), segregando etiquetas y fechas para conservarlas únicamente como metadatos descriptivos, evitando así ruidos innecesarios en el embedding.
2. **Catálogos y colecciones de registros:** Se realizó una iteración sobre cada objeto de las listas, consolidando campos de descripción y título en unidades textuales con coherencia semántica por cada registro.

Fase de limpieza y almacenamiento:

- Revisión exhaustiva para erradicar etiquetas de marcado web incrustadas en los valores de texto.
- Exclusión de ficheros auxiliares o catálogos carentes de análisis narrativo útil.
- Tratamiento de caracteres Unicode y eliminación de redundancias espaciales.
- Vinculación de un identificador único (doc_id) y asignación del fenómeno temático correspondiente según la ruta de origen en ADL.

3. Estrategias de chunking.

3.1 XLSX

Para los archivos XLSX usamos un **chunking por fila**, que es lo ideal para datos tabulares. Cada fila del Excel es un fragmento independiente guardado como pares columna-valor. Esta estrategia cumple de forma natural con el requisito de completitud lingüística: al ser una unidad semántica completa, evitamos que una idea quede cortada a la mitad (algo que sí pasa con fragmentos por tamaño fijo de tokens). Además, identificamos que haciendo lo anterior se mantiene la relación por fila, sin embargo, en la mayoría de los casos la relación no tiene sentido sin saber su contexto, por tanto se agregó el nombre del documento a cada chunk para que mantenga su contexto.

ej *chunk*: “*título_documento_xlsx | col1: val1 | col2:val2 | ... colN: valN*”.

Cada fragmento generado conserva los campos de metadata definidos por el reto: `doc_id`, `chunk_id`, `fuelle`, `formato`, `fenómeno`, `posición`, `num_tokens` y el texto completo del fragmento, lo que garantiza su trazabilidad hasta la fila y el archivo Excel exactos de donde proviene.

3.2 PDF

El texto extraído se dividió en fragmentos de máximo 220 palabras, dejando un margen frente al límite de 250 palabras establecido por el reto.

Entre fragmentos consecutivos se conservaron 40 palabras de superposición. Esta ventana compartida ayuda a mantener el contexto cuando una idea aparece cerca del punto de corte.

Cada fragmento conserva metadata para garantizar su trazabilidad:

- Identificador del documento y del fragmento.
- Fenómeno y observatorio.
- Ruta del archivo original.
- Página inicial y final.
- Posición del fragmento dentro del documento.

3.3 HTML

Para las páginas web y artículos en línea implementamos una segmentación jerárquica y estructural orientada por oraciones. El preprocesamiento consistió en depurar el marcado no visible (atributos y etiquetas decorativas) preservando la jerarquía de encabezados, párrafos y listas.

El contenido recuperado se dividió en fragmentos de máximo 200 palabras, garantizando un margen de seguridad respecto al límite de 250 palabras del reto. Para asegurar la **completitud lingüística**, los cortes se efectuaron en fronteras oracionales completas. Con el fin de mitigar la pérdida de contexto en fuentes extensas:

- **Contextualización inicial:** Se antepuso el título de la página o sección al comienzo de cada fragmento.
- **Superposición temática:** Se incluyó un *overlap* de una oración completa para dar continuidad a bloques contiguos.

Cada fragmento almacena metadatos para su trazabilidad, incluyendo identificación del documento (`doc_id`), fenómeno, fuente original, formato HTML, posición ordinal, conteo de tokens y el texto íntegro.

3.4 JSON

Para los archivos JSON diseñamos un **chunking semántico** basado en la estructura de los objetos. Dado que el corpus incluye reportes de diversas fuentes (como Alertas Tempranas, SWF y CENIA), interpretamos el esquema para extraer y concatenar los campos narrativos fundamentales (`title`, `excerpt`, `body_paragraphs`, `abstract`). Los metadatos descriptivos se aislaron para evitar ruido en el embedding, preservándose solo como atributos del documento.

El cuerpo narrativo se segmentó en unidades de entre 200 y 220 palabras, respetando las fronteras de oraciones y párrafos. En catálogos o listas, cada registro se procesó como una unidad independiente enriquecida con su contexto: “*título_artículo | resumen | cuerpo*”.

Cada *chunk* conserva su metadata asociada, detallando el identificador único, fenómeno temático, archivo fuente provisto por ADL, formato original, índice de posición y número de tokens estimados.

4. Encoders Seleccionados

4.1 PDF

Para representar semánticamente los fragmentos se usó `intfloat/multilingual-e5-base`, un encoder multilingüe de Sentence Transformers que funciona con español, inglés y portugués, los tres idiomas presentes en el corpus.

El modelo genera vectores de 768 dimensiones. Estos vectores se normalizaron y se almacenaron en un índice FAISS `IndexFlatIP`.

4.2 HTML

Para representar semánticamente los fragmentos de páginas web se extrajo el texto visible eliminando el marcado HTML y se utilizó **intfloat/multilingual-e5-base**, un encoder multilingüe de Sentence Transformers con soporte nativo en español, inglés y portugués. Este modelo, de 768 dimensiones, permite un equilibrio óptimo entre densidad semántica y eficiencia computacional, cumpliendo con la licencia MIT y las restricciones de arquitectura encoder pura.

4.3 JSON

Para los artículos y registros estructurados en JSON se seleccionaron y concatenaron los campos narrativos (title, body_paragraphs, sections, abstract), separando los metadatos descriptivos. Al igual que con HTML, se utilizó **intfloat/multilingual-e5-base** para garantizar que la similitud vectorial capture fielmente las relaciones conceptuales entre las fuentes y las consultas de evaluación.

4.4 XLSX y Mapas.

Como el proyecto exige soporte multilingüe (Sección 4.3) y el corpus incluye español, inglés y portugués, elegimos **BAAI/bge-m3**. Este modelo mantiene la potencia de búsqueda de la familia BGE y funciona de forma nativa en los tres idiomas, evitando perder coincidencias en otros idiomas. Tiene una dimensionalidad de 1024, este tamaño nos permite una mejor representación de los vectores, eliminando la ambigüedad dentro de lo posible dentro del análisis de los chunks poco extensos.

5. Índice FAISS

Para todos los encoders se utilizó el índice IndexFlatIP porque garantiza resultados 100% exactos mediante búsqueda directa por producto interno. El volumen de datos no fue lo suficientemente grande como para usar índices aproximados como *IVFFlat* o *HNSW*, ni sacrificar precisión por velocidad.

El procesamiento de los archivos PDF produjo los siguientes resultados:

- 759 registros PDF revisados.
- 757 documentos válidos procesados.
- 2 archivos inválidos identificados.
- 36.822 páginas analizadas.
- 34.364 páginas resueltas principalmente mediante extracción digital.
- 2.458 páginas recuperadas mediante OCR.

- Aproximadamente 13,3 millones de palabras recuperadas.

El texto recuperado se dividió posteriormente en 74.473 fragmentos. Cada fragmento contiene máximo 220 palabras y conserva su relación con:

- El documento de origen.
- Las páginas de procedencia.
- El fenómeno.
- El observatorio.
- La ubicación original del archivo.

El procesamiento de los archivos XLSX produjo los siguientes resultados:

- 4 archivos Excel procesados (uno por fenómeno).
- 8901 filas/chunks generados en total.
- Cada fragmento corresponde a exactamente una fila del dataset original.

6. Construcción del Grafo de Conocimiento

Como componente adicional se implementó `construirGrafo.py`, un script que construye un grafo de conocimiento a partir de los fragmentos almacenados en los archivos `metadata.jsonl` de las bases vectoriales.

El programa utiliza el modelo multilingüe Babelscape/wikineural-multilingual-ner para identificar personas, organizaciones, lugares y otras entidades en textos en español, inglés y portugués. El procesamiento se realiza por lotes y utiliza CUDA con precisión `float16` cuando hay una GPU NVIDIA disponible; de lo contrario, puede ejecutarse en CPU.

Cada entidad se convierte en un nodo y conserva los `doc_id` y `chunk_id` donde fue encontrada. Las entidades se normalizan para evitar duplicados causados por diferencias de mayúsculas, espacios o puntuación. Las relaciones se crean cuando dos entidades aparecen en el mismo fragmento. Si se encuentra un verbo relevante como desarrolla, regula, controla o afecta, este se utiliza como relación, pero en caso contrario se asigna `relacionado_con`. Por esta razón, las relaciones representan principalmente coapariciones y no necesariamente hechos comprobados.

Finalmente, el grafo dirigido se construye con NetworkX y se exporta como `entrega/grafos/grafos.graphml`, un formato compatible con NetworkX, Neo4j y Gephi.

